

Cleaning and Transforming Data

Feature engineering and transparent preparation



Prof. Gabriel Spadon, PhD

Faculty of Computer Science
spadon@dal.ca

Prepare a Table You Can Explain

- Diagnose missing values, variable types, and units.
- Create a derived variable and compare two discretization rules.
- Preserve raw monitoring records and document each cleaning decision.

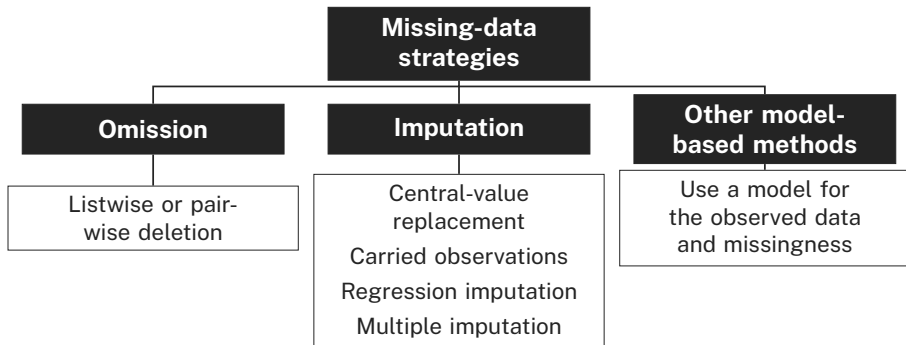
Lab 02 uses these preparation skills. Model-based sampling and PCA appear later in this deck as optional extensions.

Missing Data Mechanisms

Mechanism	What missingness can depend on
MCAR	Neither observed nor unobserved values.
MAR	Observed values; after conditioning on them, not the missing values.
MNAR	Unobserved values still matter after conditioning on observed information.

A mechanism describes the recording process. A blank cell alone does not identify why its value is missing.

Alternative Ways to Handle Missing Values



These are alternatives. Keep the original values and record the assumption behind a chosen method.

A Transparent Median Replacement

R

```
raw <- data.frame(value = c(2, NA, 8, 12))
observed <- raw[!is.na(raw$value), , drop = FALSE]
cleaned <- raw
centre <- median(observed$value)
cleaned$was_missing <- is.na(raw$value)
cleaned$value[cleaned$was_missing] <- centre
cleaned
# An illustrative choice, not a recovered measurement.
```

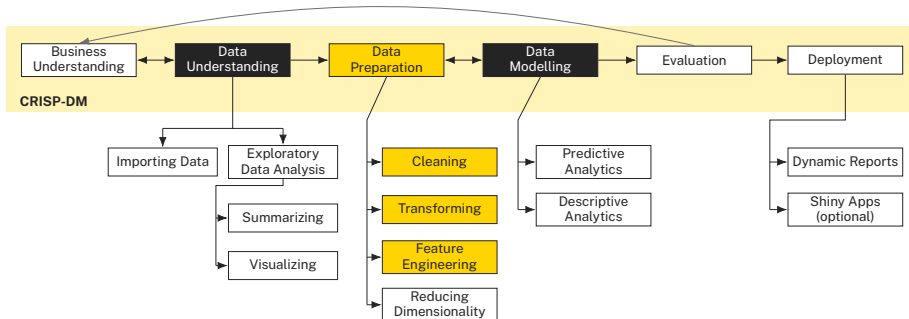
Python

```
import pandas as pd

raw = pd.DataFrame({"value": [2, None, 8, 12]})
observed = raw.dropna(subset=["value"])
cleaned = raw.copy()
centre = observed["value"].median()
cleaned["was_missing"] = raw["value"].isna()
cleaned["value"] = cleaned["value"].fillna(centre)
print(cleaned)
```

The median of 2, 8, and 12 is 8. The replacement is flagged; the raw table remains available. This rule is not a default for Lab 02.

Data Preparation in the Course Workflow



Summaries and plots help check preparation decisions. The workflow can return to earlier stages.

Four Problems, Four Responses

Variables have incompatible scales → **Data transformation**

A useful ratio needs its own variable → **Feature engineering**

Records contain unavailable or invalid values → **Data cleaning**

Too many variables obscure an analysis → **Dimensionality reduction**

Build a Variable with Meaningful Units

A derived variable combines existing measurements.

$$\text{BMI} = \frac{\text{mass in kg}}{(\text{height in m})^2}$$

For 82 kg and 178 cm, first convert the height to metres.

$$82/1.78^2 = 25.88 \text{ kg/m}^2$$

The unit conversion is part of the calculation. BMI is used here to illustrate arithmetic, not to make a clinical judgement.

Keep the Source Beside the Result

R

```
measurements <- data.frame(  
  mass_kg = c(82, 61, 95),  
  height_cm = c(178, 165, 181))  
measurements$bmi <- measurements$mass_kg /  
  (measurements$height_cm / 100)^2  
round(measurements$bmi, 2)  
# 25.88 22.41 29.00
```

Python

```
import pandas as pd  
  
measurements = pd.DataFrame({  
    "mass_kg": [82, 61, 95],  
    "height_cm": [178, 165, 181]})  
measurements["bmi"] = measurements["mass_kg"] / (  
    measurements["height_cm"] / 100)**2  
print(measurements["bmi"].round(2).to_list())  
# [25.88, 22.41, 29.0]
```

Columns from the same table refer to the same rows. R vector arithmetic uses position; pandas Series arithmetic aligns index labels.

Three Different Things You Can Reduce

Fewer variables

Select original features
Project into new
coordinates
Linear (PCA) or nonlinear

Fewer observations

Random sample
Stratified sample
Incremental sample size

Fewer distinct values

Bin numeric values
Combine categories
only with a clear reason

Each reduction discards something. Choose it for an analytical reason, not because the resulting file is smaller.

Sampling Changes Who Is Represented

R

```
data <- data.frame(row_id = 1:10,  
  site = rep(c("A", "B"), each = 5))  
set.seed(3141)  
idx <- sample(seq_len(nrow(data)), size = 6)  
random_sample <- data[idx, ]  
# Draw three rows from each site.  
idx_by_site <- split(  
  seq_len(nrow(data)), data$site)  
chosen <- unlist(  
  lapply(idx_by_site, sample, size = 3))  
stratified_sample <- data[chosen, ]  
table(stratified_sample$site)  
# A 3, B 3
```

Python

```
import pandas as pd  
  
data = pd.DataFrame({  
  "row_id": range(1, 11),  
  "site": ["A"]*5 + ["B"]*5})  
random_sample = data.sample(  
  n=6, random_state=3141)  
# Draw three rows from each site.  
stratified_sample = data.groupby("site").sample(  
  n=3, random_state=3141)  
print(stratified_sample["site"].value_counts())  
# A 3, B 3
```

Both samples contain six rows. Stratification fixes three per site. Equal seeds do not imply identical R and Python samples.

Standardize Scale Without Changing Shape

Subtract the mean, then divide by the standard deviation.

$$z_i = \frac{x_i - \bar{x}}{s_x}$$

For $x = 2, 4, 4, 5, 7, 9, 12, 41$, $\bar{x} = 10.5$
and $s_x = 12.72792$.

x_i	$x_i - \bar{x}$	z_i
2	-8.5	-0.668
4	-6.5	-0.511
7	-3.5	-0.275
12	1.5	0.118
41	30.5	2.396

Standardized values have no units. The extreme observation remains extreme; scaling does not remove skewness or make the data normal.

State Which Standard Deviation You Used

R

```
x <- c(2, 4, 4, 5, 7, 9, 12, 41)
z <- scale(x)
attr(z, "scaled:center") # 10.5
attr(z, "scaled:scale")  # 12.72792
round(z[1], 4)           # -0.6678
scale(c(x, NA))[1]       # -0.6678231
# Only the last output is NA; observed values remain.
```

Python

```
import numpy as np
from sklearn.preprocessing import StandardScaler

x = np.array([2, 4, 4, 5, 7, 9, 12, 41.])
s = StandardScaler().fit(x.reshape(-1, 1))
print(s.mean_[0])                # 10.5
print(round(s.scale_[0], 4))      # 11.9059
print(round(s.transform(x[:, None])[0, 0], 4))
# -0.7139; use ddof=1 for the R convention:
print(round(((x - x.mean()) / x.std(ddof=1))[0], 4))
# -0.6678
```

R uses the sample divisor $n - 1$; StandardScaler uses n . Both fit using observed values and preserve missing positions. A constant column needs an explicit decision before it is scaled.

Boundaries Answer Different Questions

Equal-width

Boundaries are 9.75 units apart.

Interval	Count	Members
[2, 11.75]	6	2,4,4,5,7,9
(11.75, 21.5]	1	12
(21.5, 31.25]	0	none
(31.25, 41]	1	41

Equal-frequency

Boundaries follow the quartiles.

Interval	Count	Members
[2, 4]	3	2, 4, 4
(4, 6]	1	5
(6, 9.75]	2	7, 9
(9.75, 41]	2	12, 41

Both discard within-bin differences. Equal-width retains equal numerical spacing between boundaries; equal-frequency aims for similar counts, subject to ties.

Set the Edges and Check the Counts

R

```
x <- c(2, 4, 4, 5, 7, 9, 12, 41)
edges <- seq(min(x), max(x), length.out = 5)
width_bins <- cut(x, breaks = edges,
                  include.lowest = TRUE)
quantiles <- quantile(x, probs = seq(0, 1, 0.25))
stopifnot(length(unique(quantiles)) == 5)
freq_bins <- cut(x, breaks = quantiles,
                 include.lowest = TRUE)
table(width_bins) # 6 1 0 1
table(freq_bins)  # 3 1 2 2
```

Python

```
import numpy as np
import pandas as pd

x = np.array([2, 4, 4, 5, 7, 9, 12, 41.])
edges = np.linspace(x.min(), x.max(), 5)
width_bins = pd.cut(
    x, bins=edges, include_lowest=True)
freq_bins = pd.qcut(x, q=4)
print(pd.Series(width_bins).value_counts(
    sort=False))
print(pd.Series(freq_bins).value_counts(
    sort=False))
```

Include the minimum in the first interval. Repeated quantiles need fewer justified bins or the original numeric variable; do not split identical values across categories.

Encode Categories, Not an Order

R

```
site <- factor(c("A", "B", "A", "C"))
indicators <- model.matrix(~ site - 1)
indicators
# Three columns, one for each site; no intercept.
```

Python

```
import pandas as pd

site = pd.Series(["A", "B", "A", "C"])
indicators = pd.get_dummies(site, dtype=int)
print(indicators)
# Three columns, one for each site.
```

Each row has one indicator equal to 1. Coding site A, B, C as 1, 2, 3 would suggest an order and equal spacing that these labels do not have. Lab 02 does not require a model or indicator encoding.

Four Different Value Statuses

Raw field	Status	Interpretation
0	measured	Exact recorded zero in the stated unit.
blank	missing	No usable measurement was recorded.
<0.1	censored	Below 0.1 mg/L; retain 0.1 as a limit, not a value.
oops	invalid	Unexpected token; investigate the original record.
0.24	measured	Exact recorded value, subject to provider quality flags.

These are illustrative readings in mg/L. A reporting limit bounds an unobserved measurement; it is not the measurement itself. Keep quality flags and check the provider's definitions.

Preserve the Record, Add an Interpretation

```
raw <- data.frame(  
  value_text = c("0", "", "<0.1", "oops", "0.24"),  
  unit = rep("mg/L", 5),  
  stringsAsFactors = FALSE)  
cleaned <- raw  
cleaned$status <- c(  
  "measured", "missing", "censored",  
  "invalid", "measured")  
cleaned$value_mg_L <- c(0, NA, NA, NA, 0.24)  
cleaned$limit_mg_L <- c(NA, NA, 0.1, NA, NA)  
stopifnot(identical(  
  raw$value_text, cleaned$value_text))  
# These statuses follow the stated rules.  
# Original strings and units remain.
```

Preserve the Record, Add an Interpretation

```
import pandas as pd

raw = pd.DataFrame({
    "value_text":
        ["0", "", "<0.1", "oops", "0.24"],
    "unit": ["mg/L"]*5})
cleaned = raw.copy()
cleaned["status"] = [
    "measured", "missing", "censored",
    "invalid", "measured"]
cleaned["value_mg_L"] = [
    0, None, None, None, 0.24]
cleaned["limit_mg_L"] = [
    None, None, 0.1, None, None]
assert raw["value_text"].equals(
    cleaned["value_text"])
```

Validated Conversion in R

```
text <- trimws(raw$value_text)
missing <- text == "" # This provider defines blank as missing.
censored <- grepl("^<[0-9]+([.][0-9]+)?$", text)
measured <- grepl("[0-9]+([.][0-9]+)?$", text)
cleaned$status <- ifelse(missing, "missing", ifelse(censored,
  "censored", ifelse(measured, "measured", "invalid")))
cleaned$value_mg_L <- NA_real_
cleaned$value_mg_L[measured] <- as.numeric(text[measured])
cleaned$limit_mg_L <- NA_real_
cleaned$limit_mg_L[censored] <- as.numeric(sub("^<", "", text[censored]))
cleaned$value_u_g_L <- 1000 * cleaned$value_mg_L
```

Continue from the raw and cleaned copies. These example rules recognize whole unsigned decimals and less-than limits; unrecognized tokens stay invalid. The notes explain the patterns and provide the complete script.

Validated Conversion in Python

```
text = raw["value_text"].str.strip()
missing = text.eq("") # This provider defines blank as missing.
censored = text.str.fullmatch(r"<[0-9]+([.][0-9]+)?")
measured = text.str.fullmatch(r"[0-9]+([.][0-9]+)?")
cleaned["status"] = "invalid"
for mask, label in [(missing, "missing"), (censored, "censored"), (measured, "measured")]:
    cleaned.loc[mask, "status"] = label
cleaned["value_mg_L"] = pd.to_numeric(text.where(measured))
cleaned["limit_mg_L"] = pd.to_numeric(text.where(censored).str.removeprefix("<"))
cleaned["value_ug_L"] = 1000 * cleaned["value_mg_L"]
```

Continue from the raw and cleaned copies. These example rules recognize whole unsigned decimals and less-than limits; unrecognized tokens stay invalid. The notes explain the patterns and provide the complete script.

Check Meaning Before Arithmetic

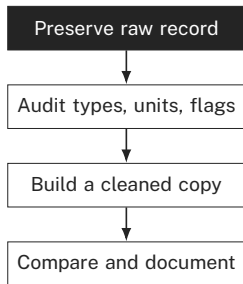
- Keep the downloaded file unchanged and preserve original strings.
- Identify station, time, parameter, unit, and provider quality flags.
- Separate censored limits from exact measurements; investigate invalid tokens.
- Convert compatible units only with a stated rule, such as $1 \text{ mg/L} = 1000 \mu\text{g/L}$.
- Report missing and censored counts beside summaries of usable measurements.

Flagging an unusual reading is a reason to investigate. It is not evidence that the reading should be deleted.

One Record, a Documented Cleaning Path

1. Read one approved monitoring file and identify its observational unit.
2. Audit missing values, reporting limits, types, units, and quality flags.
3. Build a cleaned copy with a reason for each changed or excluded value.
4. Compare summaries before and after; retain the raw record.
5. If useful, compare two binning rules on one suitable numeric variable.

The small worked examples are ungraded practice. The Lab 02 handout defines the assessed work and its rubric.



Lab 02 | Cleaning water and air quality data

Build a transparent analytical table from one official monitoring record.

Carry forward

Week 01 summaries, units, missingness, and reporting limits.

Start in class

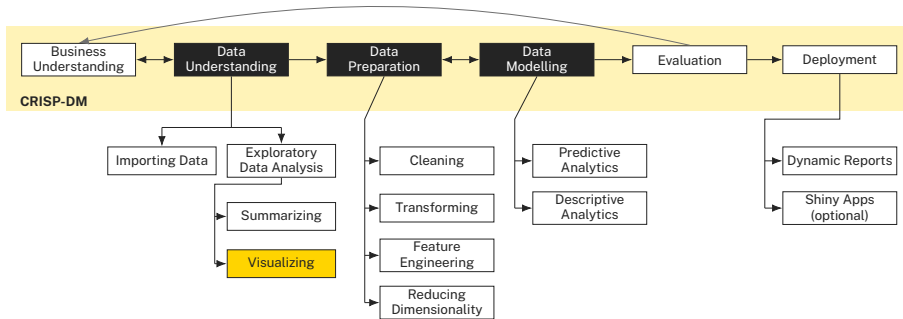
Choose one verified dataset. Work through the selection checklist, then set up the R or Python analysis.

One assessment, two languages

Submit either R or Python.
Both paths use the same evidence and rubric.

`week-02/Assessment/lab-02`

Return to the Data Through Visualization



Plots help reveal whether the summaries and cleaning choices represent the observations faithfully.

Sampling for Models and Projecting Variables

- Model-based sampling uses separate observations to compare fitted models.
- PCA projects numeric measurements onto new directions of variation.
- These examples introduce vocabulary for later classes and optional reading.

The following extensions are not required for Lab 02. The core activity ends with the documented preparation workflow.

Three Roles for Observations

- **Training** observations fit a model and its data-dependent transforms.
- **Validation** observations guide choices such as sample size or feature count.
- **Test** observations support a final evaluation after those choices are fixed.

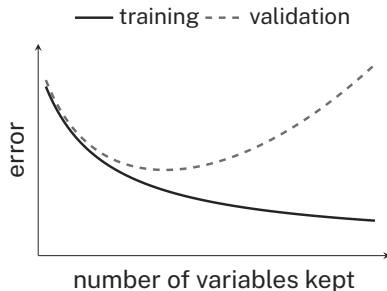
Refit neither a scaler nor PCA on validation or test rows. Related observations from the same station, person, or period may need to stay together. Later classes develop these evaluation designs.

Why a Simpler Representation Can Help

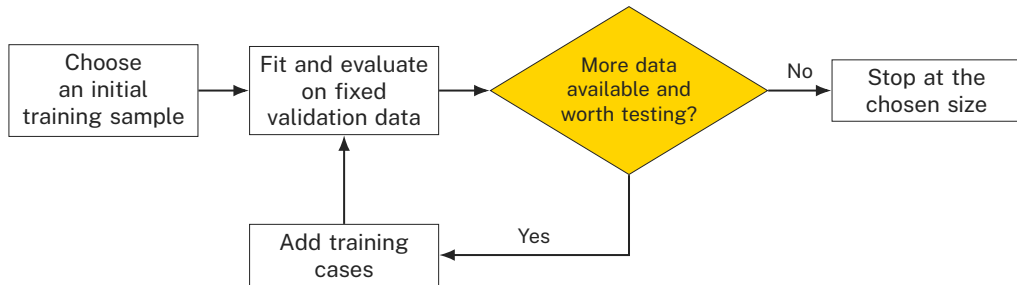
Fewer features can reduce computation and simplify interpretation.

Overfitting occurs when a model follows training-specific variation and performs poorly on new observations.

A validation comparison can reveal that adding features stops helping.

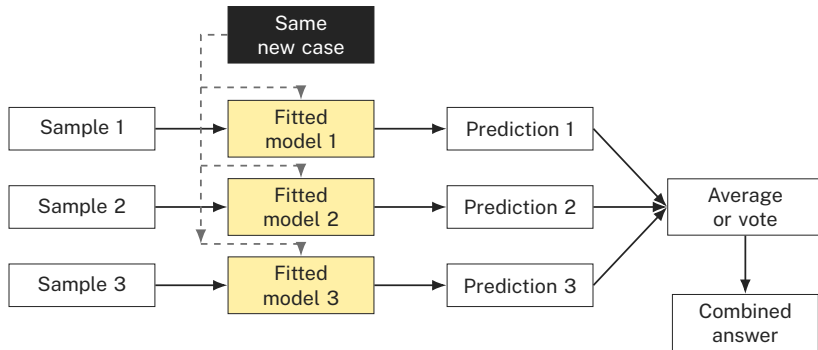


Incremental Sampling



Compare on the same validation set with a planned tolerance and repeatability check. One noisy flat increment is not proof that more data cannot help.

Combine Predictions from Several Models



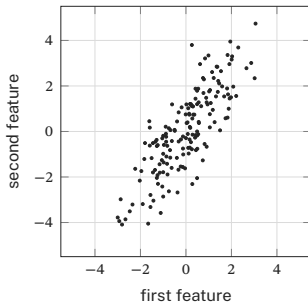
Each model predicts the same new case. Average numeric predictions or vote among class predictions. Bagging uses bootstrap resamples of the training data.

A Correlated Cloud Has a Long Direction

PCA finds directions carrying large variance.

If most variation lies along one direction, one coordinate can approximate a two-variable cloud.

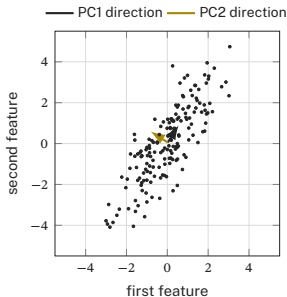
Variance does not measure relevance to a particular outcome.



Project Onto Perpendicular Directions

PC1 carries the most variance.
PC2 is perpendicular to PC1
and carries the remaining
variance in this two-feature
example.

The arrows show unit
directions. Their lengths do
not encode explained
variance.



Loadings Define a New Coordinate

	PC1	PC2	PC3	PC4
Sepal.Length	0.5210659	-0.3774176	0.7195664	0.2612863
Sepal.Width	-0.2693474	-0.9232957	-0.2443818	-0.1235096
Petal.Length	0.5804131	-0.0244916	-0.1421264	-0.8014492
Petal.Width	0.5648565	-0.0669420	-0.6342727	0.5235971

Each column is a unit-length direction. Here the original variables are the four standardized iris measurements. PC1 contrasts sepal width with the other three measurements; PC2 is dominated by sepal width.

Use Standardized Variables in the Formula

Let z_{SL} , z_{SW} , z_{PL} , z_{PW} denote the four centered, sample-standardized values.

$$PC1 = 0.5211z_{SL} - 0.2693z_{SW} + 0.5804z_{PL} + 0.5649z_{PW}.$$

The first iris observation has PC1 score -2.2571 . Multiplying the raw centimetre measurements by these weights would give a different, incorrect value for this standardized analysis.

Scaling is a choice. We standardize here to compare variability across measurements; covariance PCA can be appropriate when original scales should be retained.

Fit the Same Iris Analysis in Both Languages

R

```
features <- iris[, 1:4]
p <- prcomp(features, scale. = TRUE)
round(p$rotation, 4)          # one component per column
round(p$x[1, 1], 6)          # -2.257141
round(p$sdev^2 / sum(p$sdev^2), 4)
# 0.7296 0.2285 0.0367 0.0052
```

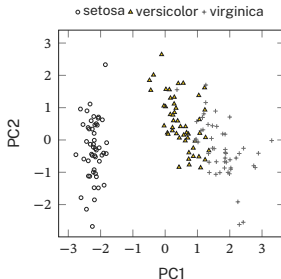
Python

```
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

features = load_iris(as_frame=True).data
Z = StandardScaler().fit_transform(features)
p = PCA().fit(Z)
print(p.components_.round(4)) # one component per row
print(p.explained_variance_ratio_.round(4))
# [0.7296, 0.2285, 0.0367, 0.0052]
```

The species label is excluded. R and Python can flip component signs. The loadings and variance proportions agree up to sign; scores differ by the sample-versus-population scaling convention.

Locate Observations With Their Scores



The shapes identify species after fitting. Visible separation describes this dataset; it does not establish predictive accuracy on new observations.

Plot the First Two Scores

R

```
features <- iris[, 1:4]
p <- prcomp(features, scale. = TRUE)
png("iris-pca-R.png", width = 700, height = 700)
plot(p$x[, 1:2], pch = as.integer(iris$Species),
     xlab = "PC1", ylab = "PC2", asp = 1)
legend("topright", legend = levels(iris$Species),
     pch = 1:3, bty = "n")
dev.off()
```

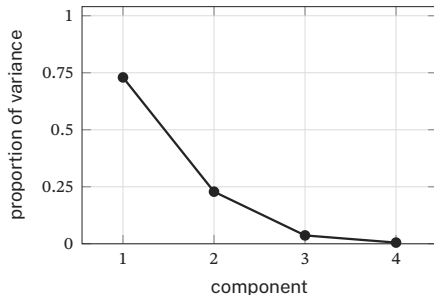
Python

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
data = load_iris()
Z = StandardScaler().fit_transform(data.data)
scores = PCA().fit_transform(Z)
for k, marker in enumerate(["o", "^", "+"]):
    points = scores[data.target == k]
    plt.scatter(
        *points[:, :2].T, marker=marker,
        label=data.target_names[k])
plt.xlabel("PC1"); plt.ylabel("PC2")
plt.axis("equal"); plt.legend()
plt.savefig("iris-pca-Python.png")
plt.close()
```

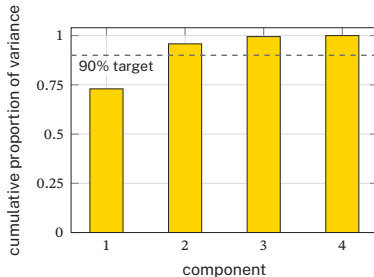
Variance Carried by Each Component

The first component carries 72.96% of the variance; the second carries 22.85%.

Later components carry progressively less variance.



Retain Enough Variance for Your Purpose



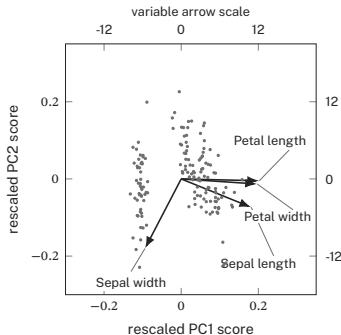
Two components retain 95.81%. They exceed an illustrative 90% target. That target is a choice, not a guarantee that useful low-variance information can be discarded.

A Biplot Combines Points and Arrows

Points are rescaled scores.
Arrows describe the original variables in the same two-component view.

The top and right scales apply to arrows; bottom and left apply to points.

Arrow angles approximate relationships only as well as this projection represents the variables.



A Biplot Needs an Explicit Display Convention

R

```
p <- prcomp(iris[, 1:4], scale. = TRUE)
# R's default biplot rescaling.
lam <- p$sdev[1:2] * sqrt(nrow(iris))
points <- sweep(p$x[, 1:2], 2, lam, "/")
arrows <- sweep(p$rotation[, 1:2], 2, lam, "*")
```

Python

```
Z = (features - features.mean()) / (
    features.std(ddof=1))
p = PCA().fit(Z)
lam = np.sqrt(p.explained_variance_[0:2] * len(Z))
points = p.transform(Z)[0:, 0:2] / lam
arrows = p.components_[0:2].T * lam
```

Both compute rescaled scores and loadings from sample-standardized iris measurements. The notes contain the complete plotting examples. The Python display uses a stated arrow factor of 1/60, so it illustrates the same quantities without claiming identical R drawing defaults.

Documentation for the Examples

- [R: scale](#) and [Python: StandardScaler](#) explain centering, divisors, and missing values.
- [R: cut](#) and [Python: qcut](#) document interval boundaries and repeated quantiles.
- [R: prcomp](#) and [Python: PCA](#) support the optional projection example.

These references are optional follow-up. The lecture notes also list the original PCA and biplot papers and the missing-data references.